Transfer Functions, Poles and Zeros

Contents

1. Overview . . . 2
2. The transfer function: a system as one fraction . . . 4
   - 2.1 An exponential goes in, the same exponential comes out . . . 4
   - 2.2 From the differential equation . . . 6
   - 2.3 From the state-space model . . . 6
3. Poles and zeros . . . 7
   - 3.1 Poles: where the fraction blows up . . . 8
   - 3.2 Zeros: where the fraction vanishes . . . 9
4. BIBO stability: which poles are safe . . . 11
5. Block-diagram algebra: the loop's fraction from its parts . . . 12
6. How large can the integral gain be: root locus and Routh–Hurwitz . . . 15
   - 6.1 The root locus . . . 15
   - 6.2 The Routh–Hurwitz criterion . . . 17
7. Pole-zero cancellation: a slow pole hidden, not removed . . . 18
8. Non-minimum phase: the reversing robot's front swings the wrong way . . . 20
9. Time delay: a late reading . . . 22
10. SISO and MIMO: two wheels, two loops . . . 24
11. Summary . . . 27
12. Sources . . . 28
13. Key terms . . . 29

Where this fits:

- **Builds on:** Block diagram of a control loop; PID control; Second-order systems: natural frequency and damping ratio.
- **Leads to:** Frequency response (gain and phase); Loop transfer function; Sensitivity functions.
- **Compare with:** z-transform.

1. Overview

Key point: A **transfer function** (G-2999) writes a linear system, such as a robot, a controller or a whole loop, as one fraction of s, and the fractions of parts combine by simple algebra. Its **poles** (G-3000), where the fraction blows up, say whether the system is stable and what shape its reply has, and its **zeros** (G-3001) reshape that reply, so we can read a loop without solving its differential equation.

The robot can now turn with chosen numbers. The faster design of choosing the gains for the turn, a P gain of 0.4 N · m/rad and a D gain of 0.4 N · m · s/rad, twice the P gain of the tuned PD controller, turns the robot 0.5 rad in 2.92 s without passing the target. This Note keeps that faster design throughout, because it meets the 3 s turn asked for there; the rest of the chapter keeps the tuned gains, 0.2 and 0.283. The tuned pair has the same damping ratio, 1, so every result below holds for it with times 1.41 times longer, and Section 6.2 gives its own integral limit. What still breaks is any loop that is not a plain second-order one. The robot's worn right gearbox still drags, a steady **disturbance** (G-2949) torque of 0.02 N · m that turns the robot to the right. Driving straight at 0.5 m/s with it (Figure 1):

- **PD alone** holds the heading 0.05 rad (2.9°) to the right of its command, so after 15 m the robot is 0.72 m to the right of its line;
- **adding integral action** (G-2963) with an **integral gain** (G-2964) of 0.2 N · m per rad s brings the heading back to its command (the tuned PID controller uses 0.05, with its softer P and D gains);
- **an integral gain of 2** makes the robot weave from side to side, wider and wider: ±0.30 rad by 30 s.

Figure 1 runs the three loops with the dragging gearbox. Watch the blue path drift off the line, the green one come back, and the red heading swing wider each time. Too much integral found the same edge by trial: with its gains, an integral gain of 0.6 made the robot weave without end. Where exactly is the edge, and why?

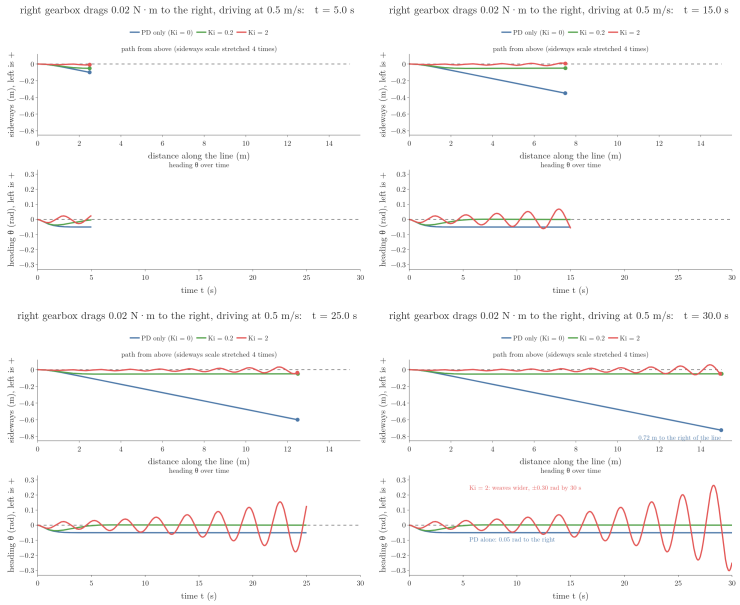


Figure 1: The opening problem: the robot drives at 0.5 m/s while a dragging right gearbox turns it to the right with 0.02 N · m. Top: the path from above, left up, the sideways scale stretched 4 times. Bottom: the heading over time, left positive. Blue, PD alone: the heading settles 0.05 rad to the right (−0.05 rad) and the robot ends 0.72 m to the right of its line after 15 m. Green, integral gain 0.2: the heading returns to 0. Red, integral gain 2: the heading weaves wider and wider, to ±0.30 rad by 30 s

The integral makes the loop third order, so the formulas of reading a second-order reply no longer apply, and solving a new third-order equation for every gain we try does not scale. This Note gives the tools that read any linear loop:

- the transfer function: a system as one fraction of s, from its differential equation or its state-space model (Section 2);
- poles and zeros: what the fraction's roots say about the reply (Section 3);
- BIBO stability: which poles let a small push grow without limit (Section 4);
- block-diagram algebra: the loop's fraction from its parts, without its equation (Section 5);
- the root locus and the Routh–Hurwitz criterion: how large the integral gain may be, 1.6 at most (Section 6);
- four traps the same tools explain on the robot: a cancelled pole that returns (Section 7), a front that swings the wrong way in reverse (Section 8), a delay (Section 9) and two wheels that drive both loops at once (Section 10).

2. The transfer function: a system as one fraction

> **Key point:** A linear system fed an exponential e^{st} gives back the same exponential times one number $G(s)$, the **transfer function** (G-2999): $1/(8s+2)$ for the robot's speed and $10/s^2$ for its heading. The transfer function comes straight from the differential equation, each derivative becoming a factor s, or from a state-space model as $C(sI-A)^{-1}B+D$.

2.1 An exponential goes in, the same exponential comes out

The gearbox problem needs the integral loop's reply for many gains, without solving a third-order equation each time. The trick that makes this possible rests on one fact: the derivative of an exponential is the same exponential times a number. For example, the derivative of $e^{-0.1t}$ is $-0.1\,e^{-0.1t}$.

So push the robot with a force that is itself an exponential, e^{st} newtons, where s is a number that sets how fast the force grows or fades. Every derivative in the robot's equation of motion only multiplies by s, so the speed can follow the same exponential, scaled by one number (FBS §9.2). Figure 2 tries it with a fading force, $s=-0.1$, and a growing one, $s=0.1$. Watch the purple ratio of speed to force: it never changes.

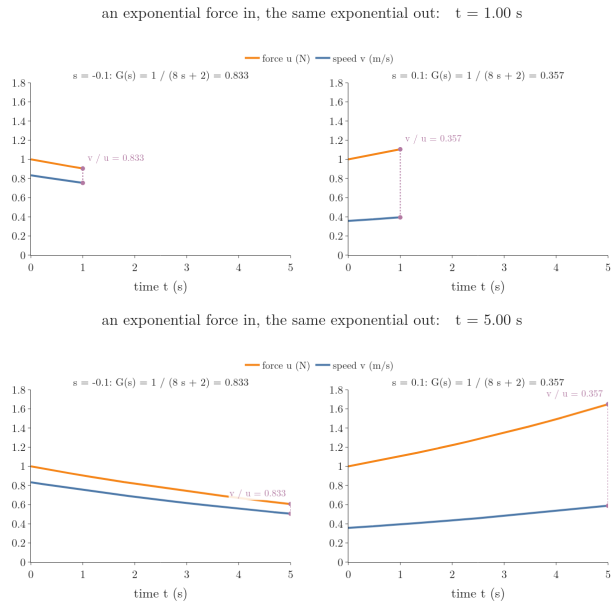


Figure 2: An exponential force in, the same exponential speed out. Left: the force fades as e^(−0.1 t) and the speed is 0.833 times the force at every instant. Right: the force grows as e^(0.1 t) and the speed is 0.357 times the force. Each robot starts at the matching speed; the purple bar and its label give the ratio of speed (m/s) to force (N), which stays at G(s) = 1/(8 s + 2)

The symbols for the robot's speed, as in the speed loop:

- m is the mass, 8 kg, and b the rolling drag, 2 N · s/m;

- u is the drive force, in N, here e^{st};
- v is the speed, in m/s, and $\dot{v}$ its rate of change;
- s is the rate of the exponential, per second: -0.1 for the fading force; it may also be a complex number, which gives a swinging exponential;
- G is the number the force is multiplied by, in m/s per N.

Put the guess, a speed of G times the force, into the equation of motion with $s = -0.1$:

$$8\,\dot{v} + 2\,v = e^{-0.1t}$$

$$8\,(-0.1\,G\,e^{-0.1t}) + 2\,G\,e^{-0.1t} = e^{-0.1t}$$

$$(-0.8 + 2)\,G = 1$$

$$G = 1/1.2 = 0.833$$

The same steps for any s:

$$m\dot{v} + b\,v = e^{st}$$

$$ms\,Ge^{st} + b\,Ge^{st} = e^{st}$$

$$(ms + b)\,G = 1$$

$$G(s) = \frac{1}{ms + b} = \frac{1}{8s + 2}$$

The function $G(s)$ that gives the output's multiple of an exponential input e^{st} is the system's **transfer function** (G-2999) (FBS §9.2). Any input can be written as a sum of exponentials, which is what the Laplace transform does: it turns a derivative into a factor s, so for a general input with the system starting at rest, the transfer function is the transformed output divided by the transformed input (FBS §9.3). At $s = 0$ the exponential is a constant force, and $G(0)$ is $1/2$: a steady 1 N gives a steady 0.5 m/s, the **DC gain** (G-2989), the final output per unit of a step input.

2.2 From the differential equation

Section 2.1's steps work for any linear equation: each derivative becomes a factor s. For the heading, with the yaw inertia I of 0.1 $\mathrm{kg \cdot m^2}$, the turning torque τ in $\mathrm{N \cdot m}$ and the heading θ in rad, the turning law is $I\ddot{\theta} = \tau$. Two derivatives give s^2:

$$0.1\, s^2\, G = 1$$

$$G(s) = \frac{1}{0.1\, s^2} = \frac{10}{s^2}$$

The controller's parts become fractions the same way (FBS Table 9.1):

- **P part**, $K_p e_\theta$: a fixed multiple, K_p;
- **D part**, $K_d \dot{e}_\theta$: one derivative, $K_d s$;
- **I part**, K_i times the running total (integral) of e_θ: the integral of e^{st} is e^{st}/s, so K_i/s.

Here e_θ is the heading error and K_i the integral gain, in $\mathrm{N \cdot m}$ per $\mathrm{rad \cdot s}$. So a PID controller is the fraction $K_p + K_d s + K_i/s$: one more fraction to combine with the robot's.

2.3 From the state-space model

A state-space model writes a system as first-order equations for a list of numbers, its state: the state's rate of change is A times the state plus B times the input, and the output is C times the state plus D times the input. The heading's state is the heading and the turn rate, so with the torque as input and the heading as output:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 10 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D = 0$$

The first row says the heading changes at the turn rate; the second says the turn rate changes at 10 times the torque, the torque divided by 0.1. With an exponential input, every state variable is an exponential too, the state is a vector X of numbers times e^{st}, and its derivative is s times it. The steps, where I inside the brackets is the identity matrix:

$$sX = AX + B$$

$$(sI - A)\, X = B$$

$$X = (sI - A)^{-1}B$$

$$G(s) = C(sI - A)^{-1}B + D$$

The last line is the state-space formula of FBS eq. 9.4. For the heading:

$$sI - A = \begin{bmatrix} s & -1 \\ 0 & s \end{bmatrix}$$

$$(sI - A)^{-1} = \frac{1}{s^2}\begin{bmatrix} s & 1 \\ 0 & s \end{bmatrix}$$

$$(sI - A)^{-1}B = \frac{1}{s^2}\begin{bmatrix} 10 \\ 10s \end{bmatrix}$$

$$G(s) = \frac{10}{s^2}$$

the same fraction as Section 2.2. A check at $s = 2$ (notebook):

$$(2I - A)^{-1} = \begin{bmatrix} 0.5 & 0.25 \\ 0 & 0.5 \end{bmatrix}$$

$$C(2I - A)^{-1}B = 0.25 \times 10 = 2.5$$

$$10/2^2 = 2.5$$

The two routes agree, so either the equation or the state-space model gives the transfer function. The PX4 autotuner stores the drone model it fits as exactly such a fraction: three numerator and three denominator coefficients of a discrete-time transfer function, the sampled form used in digital control (PX4's `mc_autotune_attitude_control.cpp`, lines 189 and 190). What does the fraction tell us about the reply?

3. Poles and zeros

> **Key point:** The **poles** (G-3000) of a transfer function are the values of s where its denominator is zero; each adds a term e^{pt} to the reply, so a pole's real part says whether that term fades (negative) or grows (positive) and its imaginary part how fast it swings. The **zeros** (G-3001) are where the numerator is zero; they do not change stability but reshape the reply: a zero at -1 adds the reply's own rate of change and makes a critically damped turn overshoot 13.5 %.

3.1 Poles: where the fraction blows up

The closed heading loop of the faster design has the transfer function from command to heading $4/(s^2+4s+4)$ (Section 5 builds it). Its denominator is zero at $s=-2$, twice: exactly the two roots of the characteristic equation (G-2993), the ones that made the turn critically damped. The match is no accident. Write a transfer function as a numerator $N(s)$ over a denominator $D(s)$:

- the **poles** (G-3000) are the values of s where $D(s)=0$, so the fraction blows up there (Chen L6);
- the **zeros** (G-3001) are the values of s where $N(s)=0$, so the fraction is zero there (Chen L6).

At a pole, an input that is zero can still give an output: the system can follow e^{pt} on its own. So each pole p is one of the system's natural motions, a term e^{pt} in every reply (Chen L6; MR 11.2.2). In a state-space model the poles are the **eigenvalues** (G-665) of A, the roots of its **characteristic polynomial** (G-376), the determinant of $sI-A$, because the inverse in Section 2.3 divides by that determinant. For the robot's heading alone it is s^2, two poles at 0; for the PD loop, s^2+4s+4, two poles at -2 (notebook).

Poles are drawn as crosses, and zeros as circles, in the plane of complex numbers, real part across and imaginary part up: a **pole-zero map** (G-3002) (Chen L6). The imaginary axis splits it into the **left half-plane** (G-3003), where the real part is negative, and the right half-plane, where it is positive. Figure 3 moves one pole, then a pair, around that plane and draws the term they add. Watch three things: a pole further left fades faster; a pair further from the real axis swings faster; anything in the right half-plane grows.

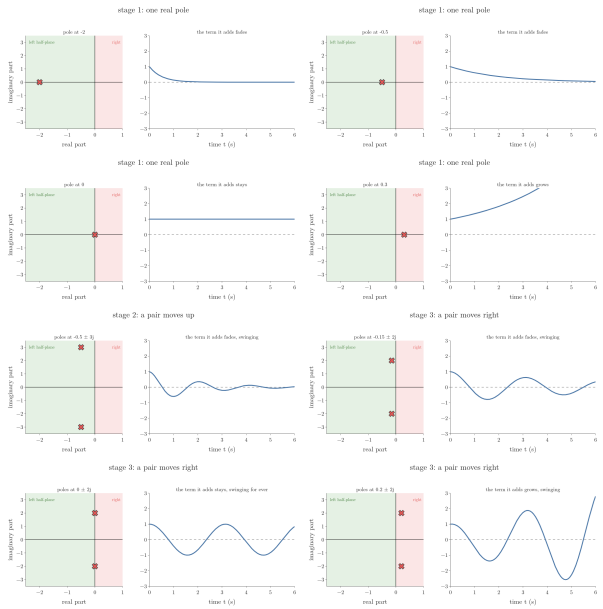


Figure 3: Where a pole sits sets the shape of what it adds to a reply. Left: the plane of complex numbers, the left half-plane green, the right half-plane red. Right: the term the pole or pair adds, starting at 1. Stage 1: one real pole moves from -2 to $+0.3$: a fast fade, a slow fade, a constant at 0, growth. Stage 2: a pair with real part -0.5 moves up from ± 1j to ± 3j: faster swings. Stage 3: the pair at ± 2j moves right from -0.5 to $+0.2$: the swing fades more slowly, lasts for ever on the imaginary axis, then grows

The rule behind Figure 3 (Chen L6), with:

- p a pole, a complex number such as $-0.5 + 2j$;
- σ its real part, per second, -0.5 here;
- ω its imaginary part, in rad/s, 2 here.

$$e^{pt} = e^{\sigma t}\, e^{j\omega t}$$

$$= e^{\sigma t}(\cos \omega t + j \sin \omega t)$$

The first factor fades when the real part is negative and grows when it is positive; the second swings at ω rad/s. A pair of poles with the same real part and opposite imaginary parts adds the real swing $e^{\sigma t} \cos \omega t$. Poles give the natural motions; what do zeros do?

3.2 Zeros: where the fraction vanishes

The faster design puts the D part on the measured turn rate, which the gyro gives. Putting it on the error instead gives the same denominator but a new numerator (Section 5 shows how):

$$\frac{0.4s + 0.4}{0.1s^2 + 0.4s + 0.4} = \frac{4s + 4}{s^2 + 4s + 4}$$

Same poles, -2 twice, so the same damping ratio, 1, and by the second-order formulas no overshoot. But the numerator is zero at $s = -1$: a zero. Figure 4 runs the 0.5 rad turn both ways. Watch the orange turn rise faster and pass the target, peaking at 0.568 rad at 1.0 s, 13.5 % past, although its poles are those of a critically damped loop.

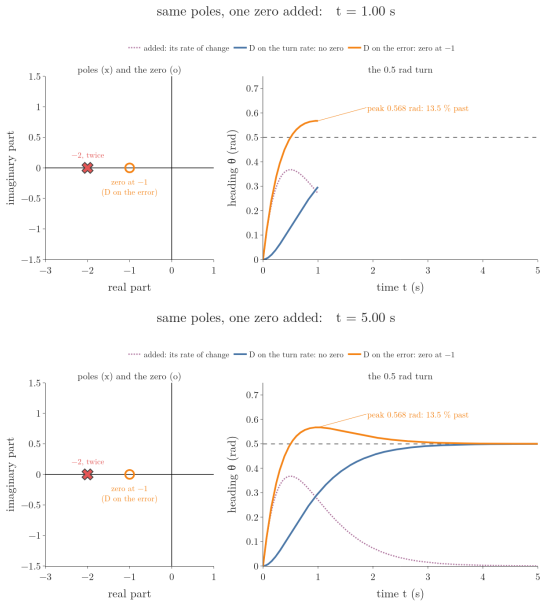


Figure 4: Same poles, one zero added. Left: the poles of both loops, -2 twice (crosses), and the zero at -1 of the loop whose D part acts on the error (circle). Right: the 0.5 rad turn. Blue: D on the measured turn rate, no zero, no overshoot. Orange: D on the error, zero at -1: it peaks at 0.568 rad at 1.0 s, 13.5 % past the target. Purple dotted: the rate of change of the blue turn, which the zero adds to it

Why a zero does this. The new numerator is $s + 1$ times the old one, 4. Multiplying by s means taking the derivative, so the new reply is the old reply plus its own rate of change (Chen L9). The old reply, from critical damping, with θ the heading in rad and t the time in s:

$$\theta = 0.5 - 0.5\,e^{-2t} - t\,e^{-2t}$$

$$\dot{\theta} = 2t\,e^{-2t}$$

$$\theta + \dot{\theta} = 0.5 - 0.5\,e^{-2t} + t\,e^{-2t}$$

Its rate of change is $(1 - t)$ times $2e^{-2t}$, zero at $t = 1$ s, where:

$$0.5 - 0.5\,e^{-2} + e^{-2}$$

$$= 0.5 + 0.5 \times 0.1353 = 0.5677 \text{ rad}$$

an overshoot of 0.1353, 13.5 % (notebook). The general rule: a zero at $-a$ adds $1/a$ times the reply's rate of change to it, so a zero near the poles speeds up the rise and can add overshoot, and a far-away zero changes little (Chen L9). The added derivative is why the second-order formulas hold only for a loop without zeros, and why the D part usually acts on the measurement; a D part on the error also jolts the torque at the step, the derivative kick. Poles and zeros together give the shape; the poles alone decide the more urgent question for the gearbox: does a push stay small?

4. BIBO stability: which poles are safe

> **Key point:** A system is **BIBO stable** (G-3004) when every bounded input gives a bounded output, which for a transfer function without delay holds exactly when every pole lies in the open left half-plane. Poles on the imaginary axis make it **marginally stable** (G-3005): it never settles but does not blow up by itself, yet a bounded input at the right rate makes it grow, so it is not BIBO stable.

The gearbox's drag is small and steady, 0.02 N · m. Will it always give a small heading error, or can a small push grow without limit? Figure 5 pushes three systems with small, bounded inputs. Watch which curves stay inside their plots.

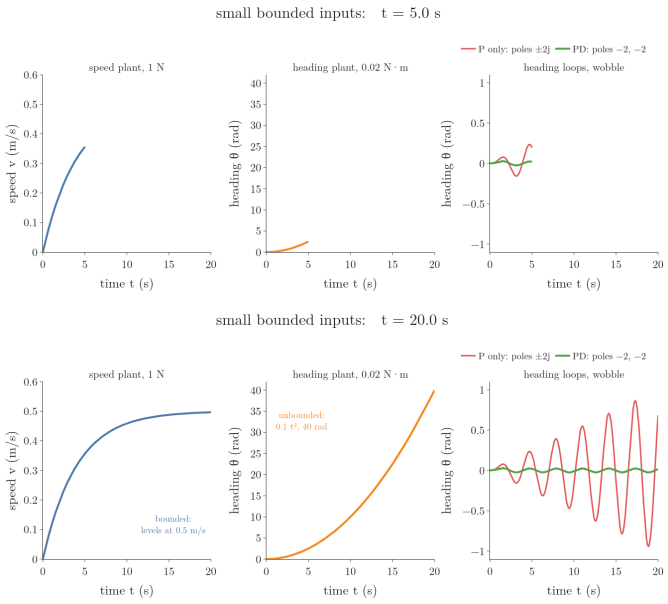


Figure 5: Small bounded inputs, drawn over 20 s. Left: the speed plant $1/(8s + 2)$, one pole at -0.25, under a steady 1 N: the speed levels off at 0.5 m/s. Middle: the heading plant $10/s^2$, two poles at 0, under a steady 0.02 N · m: the heading grows as 0.1 t^2, to 40 rad. Right: a torque wobble of 0.02 sin(2t) N · m on two heading loops: P alone, poles ±2j (red), swings wider by about 0.05 rad every second, 0.94 rad by 20 s; PD, poles −2 twice (green), stays within 0.03 rad

- **The speed plant**, pole at -0.25: a steady 1 N gives a steady 0.5 m/s, and any force up to the 4 N limit keeps the speed under 2 m/s, so its output stays bounded.
- **The heading plant alone**, two poles at 0: a steady 0.02 N · m gives an angular acceleration of 0.2 rad/s^2, so the heading grows as 0.1 t^2, 40 rad after 20 s, without limit. The robot spins faster and faster: a bounded input, an unbounded output.
- **A P-only heading loop**, gain 0.4: its closed loop is $4/(s^2+4)$, poles at $\pm 2j$, on the imaginary axis. Left alone it would swing for ever without growing. But a wobbling torque at exactly 2 rad/s, such as a wheel that is slightly out of round, pushes each swing at the right moment, like pushing a child on a swing, and the swing grows by about 0.05 rad every second.
- **The PD loop**, poles at -2 twice: the same wobble moves the heading by at most 0.03 rad (notebook).

A system is **BIBO stable** (G-3004), bounded input, bounded output, when every bounded input gives a bounded output (FBS §10.5; Chen L10). For a transfer function without delay that holds exactly when every pole has a negative real part: then every term e^{pt} fades, and a steady push only adds a steady reply (Chen L10). A system with poles on the imaginary axis, and none to the right, is **marginally stable** (G-3005): its own motion neither fades nor grows, but it is not BIBO stable, because an input at the swing's own rate makes it grow (Chen L10). With no time delay in the loop, the poles alone decide stability (Chen L6); Section 9 shows what changes when there is one.

So the gearbox question becomes a question about poles: where are the poles of the loop with an integral gain? Finding them needs the loop's transfer function.

5. Block-diagram algebra: the loop's fraction from its parts

> **Key point:** Three rules reduce any loop drawing to one fraction: blocks in series multiply, blocks in parallel add, and a feedback loop becomes the forward path divided by one plus the product around the loop. The PID heading loop reduces to $(4s+10K_i)/(s^3+4s^2+4s+10K_i)$, and every transfer function of the loop, from the command or from the gearbox's drag, shares that denominator.

We need the PID heading loop's transfer function. Writing its third-order equation and transforming it would work, but there is a shorter way: combine the fractions of its parts, the controller and the robot, by the way they are wired. A drawing of the parts as boxes and the signals as arrows is a **block diagram** (G-2950), with circles, **summing junctions** (G-2951), where signals add or subtract. Three rules combine its boxes, which together are **block-diagram algebra** (G-3006) (Chen L2; FBS §9.4):

1. **Series:** when one block's output is the next block's input, their fractions multiply.
2. **Parallel:** when one signal feeds several blocks whose outputs are added, their fractions add.
3. **Feedback:** a loop whose output is fed back and subtracted at its input becomes the forward path divided by one plus the product of all the blocks around the loop.

Why the feedback rule holds. The symbols, each the multiple of an exponential e^{st} as in

Section 2:

- R is the command and Y the output;
- C is the controller's transfer function and P the robot's.

The error is $R - Y$; the controller turns it into $C(R - Y)$ and the robot turns that into $PC(R - Y)$, which is the output (Chen L2). Numbers first, for the P-only heading loop of Section 4 at $s = 2$: the robot's fraction is 10 over 2^2, 2.5, and the controller's is 0.4, so PC is 1:

$$Y = 1 \times (R - Y)$$

$$2\,Y = R$$

$$Y/R = 0.5$$

which is that loop's fraction, $4/(s^2 + 4)$, at $s = 2$:

$$\frac{4}{2^2 + 4} = 0.5$$

The same steps in symbols:

$$Y = PC\,(R - Y)$$

$$(1 + PC)\,Y = PC\,R$$

$$\frac{Y}{R} = \frac{PC}{1 + PC}$$

The fraction from a command to the output of a closed loop is its **closed-loop transfer function** (G-3007). A sensor block H in the feedback path, such as a gyro with its own lag, would join the loop product, making the denominator $1 + PCH$; our gyro is taken as perfect, $H = 1$, until Section 9.

Figure 6 reduces the heading loop one move at a time: P and I on the error, D on the measured turn rate, the robot $10/s^2$. Watch each move replace two boxes, or a box and its loop, by one.

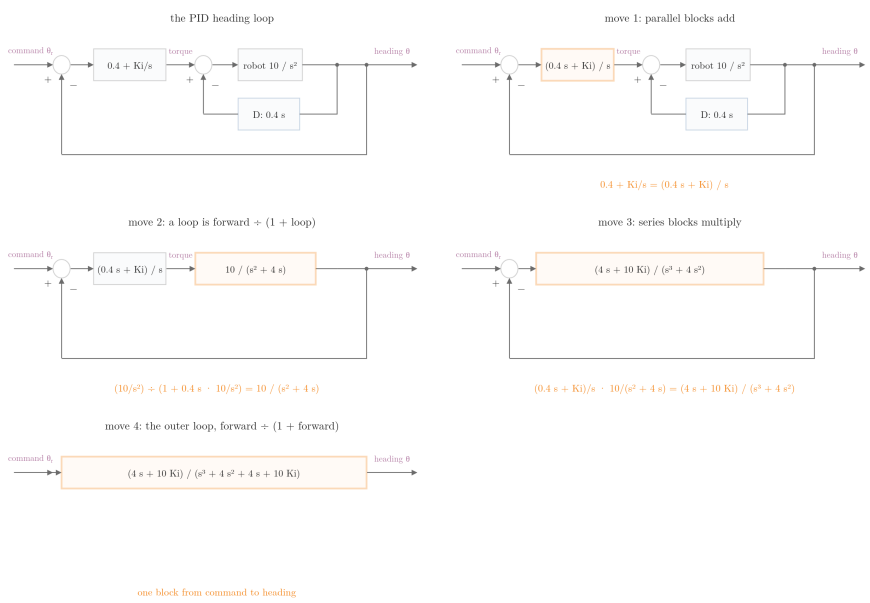


Figure 6: Block-diagram algebra on the PID heading loop, one move per frame. 0: the loop: command, the P and I parts (0.4 + Ki/s), the torque, the robot $10/s^2$, the D part 0.4 s fed back from the heading, and the heading fed back to the start. Move 1: parallel blocks add, 0.4 + Ki/s = (0.4 s + Ki)/s. Move 2: the inner loop is forward ÷ (1 + loop), $10/(s^2 + 4s)$. Move 3: series blocks multiply, $(4s + 10\ \text{Ki})/(s^3 + 4s^2)$. Move 4: the outer loop gives one block, $(4s + 10\ \text{Ki})/(s^3 + 4s^2 + 4s + 10\ \text{Ki})$

The four moves, with these symbols:

- C is the P and I parts together, from heading error to torque;
- K_i is the integral gain, in N · m per rad s, left as a symbol;
- P_1 is the robot together with its D feedback, from torque to heading;
- G_{cl} is the closed-loop transfer function, from command to heading.

$$C = 0.4 + \frac{K_i}{s} = \frac{0.4s + K_i}{s}$$

$$P_1 = \frac{10/s^2}{1 + 0.4s \cdot 10/s^2}$$

$$= \frac{10}{s^2 + 4s}$$

$$CP_1 = \frac{4s + 10K_i}{s^3 + 4s^2}$$

$$G_{cl} = \frac{CP_1}{1 + CP_1}$$

$$= \frac{4s + 10K_i}{s^3 + 4s^2 + 4s + 10K_i}$$

The notebook checks every move at three values of s. With no integral gain, $K_i = 0$, the s cancels top and bottom and G_{cl} is $4/(s^2 + 4s + 4)$, the faster design's PD loop. The gearbox's torque enters at the robot's input, so the fraction from that drag to the heading is $P_1/(1 + CP_1)$:

$$\frac{10s}{s^3 + 4s^2 + 4s + 10K_i}$$

with the same denominator. Every transfer function of one loop shares the loop's denominator, $1 + CP_1$ cleared of fractions, so the poles that shape the turn also shape the recovery from the drag. Multiplying the denominator by 0.1 gives it in the gains' own units:

$$0.1s^3 + 0.4s^2 + 0.4s + K_i$$

whose coefficients are the yaw inertia, the D gain, the P gain and the integral gain. Now the gearbox question has a precise form: for which integral gains are the three roots of that cubic in the left half-plane?

6. How large can the integral gain be: root locus and Routh–Hurwitz

> **Key point:** As the integral gain rises from 0, the three poles of the PID heading loop trace the **root locus** (G-3008): two meet near -0.67 at a gain of 0.12, split off the real axis and reach the imaginary axis at $\pm 2j$ when the gain is 1.6. The **Routh–Hurwitz criterion** (G-3009) gives that limit from the coefficients alone (4 times 4 must exceed 10 times the gain), and a gain of 0.2 removes the gearbox's error within 4.68 s.

6.1 The root locus

With the denominator of Section 5, we can find the poles for any integral gain by solving one cubic, no simulation needed. Figure 7 does it for gains from 0 to 2 and draws the path the poles trace, next to the heading's reply to the gearbox's drag. Watch the two right-hand poles: as they approach the imaginary axis the reply swings longer, and once they cross it the swing grows.

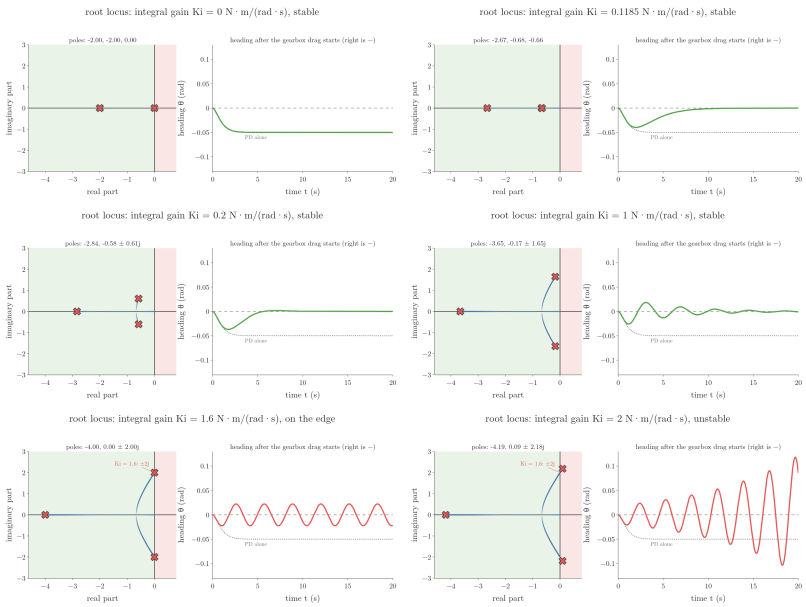


Figure 7: The root locus of the PID heading loop as the integral gain Ki rises from 0 to 2. Left: the three closed-loop poles, the roots of $0.1\ s^3 + 0.4\ s^2 + 0.4\ s + Ki$, with the path they have traced; the left half-plane is green, the right half-plane red. Right: the heading after the gearbox's 0.02 N · m drag to the right starts (right is negative); the dotted line is PD alone, 0.05 rad to the right. Ki = 0: poles 0, −2, −2. Ki = 0.1185: two poles meet near −0.67. Ki = 0.2: −2.84 and −0.58 ± 0.61j; the heading returns to 0. Ki = 1: −3.65 and −0.17 ± 1.65j; long swings. Ki = 1.6: ±2j; a swing that never dies. Ki = 2: 0.09 ± 2.18j, in the right half-plane; the swing grows

The path the closed-loop poles trace as one gain changes is the **root locus** (G-3008) (MR 11.3; Chen L13; FBS §12.5). The locus starts at the poles of the loop without that gain and shows stability, speed and swing together, for every gain at once. For the heading loop (notebook):

Integral gain	Poles	Reply to the gearbox's drag
0	0, −2, −2	stays 0.05 rad off
0.12	−2.67, two near −0.67	returns, no swing
0.2	−2.84, −0.58 ± 0.61j	returns within 0.005 rad by 4.68 s
1	−3.65, −0.17 ± 1.65j	swings for many seconds
1.6	−4, ±2j	swings for ever
2	−4.19, 0.09 ± 2.18j	swings wider and wider

The shape is the one MR 11.4 describes for adding an integral to a PD loop: the integral removes the steady error, but it adds a pole, and too much of it drags two poles across into the right half-plane (MR 11.4). A gain near the meeting point keeps the reply free of swings; the green run of Figure 1 uses 0.2, where the slow pair swings a little (imaginary part 0.61) but sits further left: its heading swings at most 0.037 rad to the right and is back within

0.005 rad by 4.68 s, against 7.39 s at 0.12 (notebook). Real controllers keep the integral gain small and limit the integral term, holding it inside fixed bounds (MR 11.4): Autoware's longitudinal PID controller uses an integral gain of 0.1 and limits the integral's output to ±0.3 (`ki: 0.1` and `max_i_effort: 0.3` in its parameter file, full path in Sources). Limiting the integral term is one of the fixes for windup, what goes wrong when the torque is at its limit.

6.2 The Routh–Hurwitz criterion

The root locus needs the roots for every gain. For the one question, "where does the loop become unstable?", there is a shortcut that needs no roots at all. Write the cubic with its first coefficient 1, by multiplying the denominator of Section 5 by 10, and name its coefficients:

- a_1, a_2 and a_3 are the coefficients of s^2, s and 1: 4, 4 and 10 times the integral gain.

$$s^3 + a_1 s^2 + a_2 s + a_3$$

The **Routh–Hurwitz criterion** (G-3009) decides from the coefficients alone whether every root has a negative real part (FBS §2.2; Chen L10). For a cubic it says (FBS eq. 2.18):

$$a_1,\ a_2,\ a_3 > 0$$

$$a_1 a_2 > a_3$$

For the heading loop:

$$4 \times 4 > 10\, K_i$$

$$K_i < 1.6$$

Check at the edge. At $K_i = 1.6$ the cubic is:

$$s^3 + 4s^2 + 4s + 16$$

Put in $s = 2j$, using $j^2 = -1$:

$$(2j)^3 + 4(2j)^2 + 4(2j) + 16$$

$$= -8j - 16 + 8j + 16 = 0$$

so $\pm 2j$ are roots: two poles sit on the imaginary axis, the never-dying swing of Figure 7, which swings at 2 rad/s (notebook). For higher orders the criterion builds a table from the coefficients and counts sign changes down its first column (Chen L10); the cubic rule is that table's answer for three roots. The gearbox question is answered: any integral gain between 0 and 1.6 is stable, 0.2 removes the 0.05 rad error within 4.68 s, and a gain of 2 weaves because two poles have crossed into the right half-plane.

The same rule explains the edge found by trial in too much integral. The tuned loop there has a P gain of 0.2 and a D gain of 0.283, so its cubic, times 10, is:

$$s^3 + 2.83\,s^2 + 2s + 10K_i$$

The rule asks for:

$$2.83 \times 2 > 10\,K_i$$

$$K_i < 0.566$$

Its integral gain of 0.6 was just past the edge, which is why that robot weaved without end. The same poles and zeros also explain four traps the robot meets next.

7. Pole-zero cancellation: a slow pole hidden, not removed

> **Key point:** A controller zero placed exactly on a plant pole hides that pole from the reply to the command, which then looks clean. The pole is still there in the reply to a push, so **pole-zero cancellation** (G-3010) of a slow pole gives a slow recovery from disturbances, and of an unstable pole a signal inside the loop that grows without limit.

The first trap is in the speed loop. Its plant, $1/(8s+2)$, has a slow pole at -0.25: a time constant of 4 s. Its PI controller from integral action, a P gain of 20 N per m/s and an integral gain of 10 N per m, is the fraction $(20s+10)/s$, with a zero at -0.5. To show cancellation we change one number: an integral gain of 5 moves that zero to -0.25, right on the plant's pole:

$$\frac{20s+5}{s} \cdot \frac{1}{8s+2}$$

$$= \frac{20(s+0.25)}{s} \cdot \frac{1}{8(s+0.25)}$$

$$= \frac{2.5}{s}$$

The loop closes, by Section 5's rule, to $2.5/(s+2.5)$: a clean first-order reply to the command, with a time constant of $1/2.5$, 0.4 s, no overshoot and no steady-state error. The slow pole seems gone; this is a common, simple design (FBS §9.5, Example 9.12).

Now put the robot on the 1 % ramp of feedback basics, which pulls it back with 0.785 N. The pull enters at the plant, and the fraction from it to the speed is the plant over one plus the loop:

$$\frac{-1/(8s+2)}{1+2.5/s}$$

$$= \frac{-s}{8(s+0.25)(s+2.5)}$$

The pole at -0.25 is back. Figure 8 runs both integral gains: the robot cruises at 0.4 m/s, the command steps to 0.5 m/s at t = 0, and from 10 s the ramp pulls. Watch the blue speed answer the command without overshoot, then recover from the ramp more slowly than the orange one.

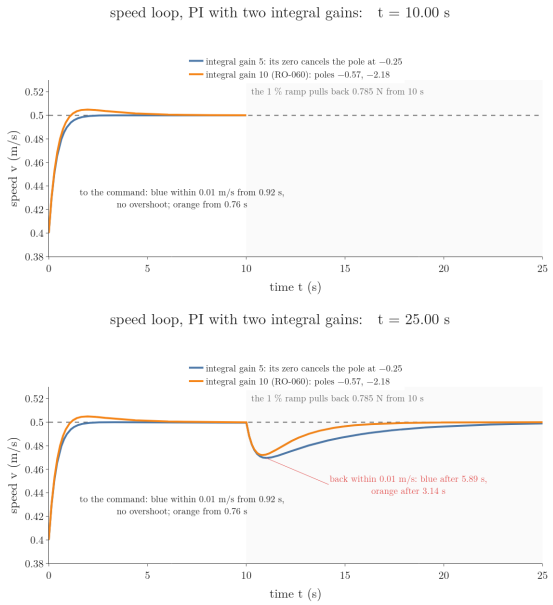


Figure 8: Pole-zero cancellation hides a slow pole. The speed loop under PI control, P gain 20 N per m/s, with integral gain 5, whose zero cancels the robot's pole at -0.25 (blue), and the speed loop's usual 10, which cancels nothing (orange). The robot cruises at 0.4 m/s; the command steps to 0.5 m/s at t = 0; from t = 10 s the 1 % ramp pulls back with 0.785 N (shaded). To the command, blue is within 0.01 m/s from 0.92 s with no overshoot, orange from 0.76 s after overshooting by 0.005 m/s. To the ramp, blue dips to 0.470 m/s and is back within 0.01 m/s after 5.89 s, orange after 3.14 s

Why the recovery is slow. Split the reply to the ramp's 0.785 N into its two poles' terms (partial fractions; notebook):

$$\text{speed change} = -0.0436\,(e^{-0.25t} - e^{-2.5t})$$

where 0.0436 is 0.785/18. The fast term fades with a time constant of 0.4 s; the slow one, $e^{-0.25t}$, with the plant's 4 s. The dip is deepest where the two terms change at the same rate:

$$0.25\,e^{-0.25t} = 2.5\,e^{-2.5t}$$

$$e^{2.25t} = 10$$

$$t = \ln 10/2.25 = 1.02 \text{ s}$$

$$-0.0436 \times (0.7743 - 0.0774)$$

$$= -0.0304 \text{ m/s}$$

so the speed falls to 0.470 m/s and then creeps back with the 4 s pole: it is back within 0.01 m/s only after 5.89 s, against 3.14 s with the speed loop's usual integral gain of 10, whose poles, −0.57 and −2.18, include no slow one. The cancelling gain wins only on the shape of the reply to the command (no overshoot, against 0.005 m/s) and loses on the ramp, because cancellation hides the slow pole from one transfer function only (FBS Example 9.12, where the cancelled design also recovers poorly from a slope).

Cancellation is dangerous when the cancelled pole is unstable. A zero at +1 placed on a plant pole at +1 gives a reply to the command that looks fine, but a signal inside the loop still contains e^t and grows without limit, and any small mismatch between the zero and the pole leaves the unstable pole in place (FBS §9.5, Example 9.12). The robot's heading plant has its two poles at 0, on the edge, which is one reason we never try to cancel them. The next trap is a zero in the wrong half of the plane.

8. Non-minimum phase: the reversing robot's front swings the wrong way

> **Key point:** A zero in the right half-plane makes a reply start in the wrong direction before it turns round; such a system is **non-minimum phase** (G-3011). A robot reversing at 0.1 m/s while turning left shows it at its front bumper, 0.1 m ahead of the axle: the zero is at +1, and the bumper first moves 0.016 m left before swinging right.

The robot backs into its charging dock, slowly, at 0.1 m/s, turning left by 0.5 rad on the way with the faster design's turn. Which way does its front bumper go?

The bumper's sideways position is the axle's sideways position plus the part due to the turn. The symbols:

- y is the sideways position of the axle centre, in m;
- y_f is the sideways position of the bumper point, in m, left positive;
- a is the bumper's distance ahead of the axle, 0.1 m;
- v is the speed, -0.1 m/s when reversing;
- θ is the heading, small, in rad.

For a small heading the axle moves sideways at the speed times the heading, and the turn alone moves the bumper sideways by a times the heading (FBS Example 10.11 uses the same kind of model for a car):

$$\dot{y} = v\,\theta$$

$$y_f = y + a\,\theta$$

Each derivative becomes s, so the axle's sideways position is $v\theta/s$, and from the heading to the bumper:

$$\frac{y_f}{\theta} = \frac{v}{s} + a$$

$$= \frac{as + v}{s}$$

The numerator is zero at $s = -v/a$. Driving forwards at 0.1 m/s the zero is at -1, in the left half-plane; reversing, at $+1$, in the right half-plane. Figure 9 runs both. Watch the purple bumper: forwards it moves left at once; backwards it first moves left, the wrong way, by 0.016 m, crosses back at 1.78 s and ends 0.145 m to the right.

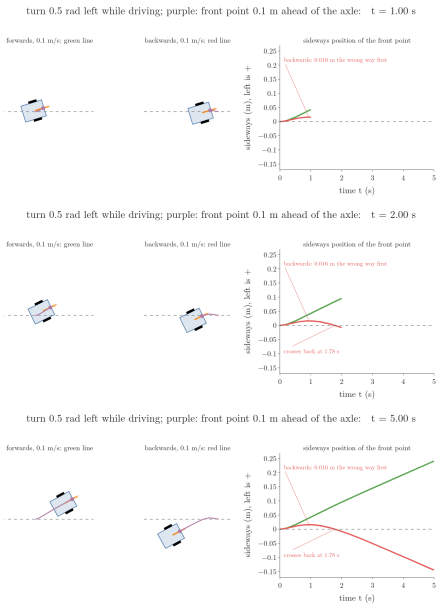


Figure 9: Non-minimum phase: the reversing robot's front point. The robot turns 0.5 rad left while driving at 0.1 m/s, forwards (left panel) or backwards (middle). The purple dot is a point on the front bumper, 0.1 m ahead of the axle, with its trail. Right: the sideways position of that point, left positive. Forwards it moves left from the start, 0.24 m after 5 s. Backwards it first moves 0.016 m left by 0.94 s, crosses back at 1.78 s and ends 0.145 m to the right

The mechanism. Of the two parts of the bumper's motion, $a\theta$ acts at once, as soon as the robot turns, while v times the running total of the heading builds up slowly. Reversing, v is negative, so the slow part has the opposite sign and wins later: the reply starts the wrong way (Chen L9, where a right-half-plane zero gives "undershoot"; FBS Example 10.11, where a car reversing shows the same "inverse response"). A system whose transfer function has a zero in the right half-plane (or a time delay, Section 9) is **non-minimum phase** (G-3011) (FBS §10.4). The name comes from the frequency response, gain and phase; what matters here is the wrong-way start, which limits how fast a loop can be made to respond, since pushing harder on a reply that starts backwards makes it start further backwards (Chen L9). The zero also depends on where we measure: at the axle, $a = 0$, it disappears (FBS Example 10.11). A late reading is the next trap.

9. Time delay: a late reading

> **Key point:** A **time delay** (G-3012) of T_d seconds has the transfer function e^{-sT_d}: it is not a fraction, so it has no poles or zeros, and "the poles decide stability" no longer covers a loop that contains one. In the heading loop a delay of 0.17 s changes almost nothing, 0.3 s makes the turn overshoot 14.6 %, and near 0.32 s the swing stops dying out.

Suppose the heading comes from a camera that reports it late, or the drive an-

swers its command late. Autoware's longitudinal controller assumes 0.17 s for this (`delay_compensation_time: 0.17`), because, its README says, the mechanism that moves a car's pedals typically has "a delay of about a hundred millisecond". How much delay can the turn take?

A signal delayed is the same signal, later. With:

- T_d the delay, in s: 0.17 s for Autoware, up to 0.33 s below.

Fed an exponential, a delay gives back:

$$e^{s(t-T_d)} = e^{-sT_d}\, e^{st}$$

so its transfer function is e^{-sT_d} (FBS §9.2, Example 9.5). The delay's transfer function is not a fraction of two polynomials: it has no poles and no zeros, and Section 4's rule, "the poles alone decide stability", was for loops without delay (Chen L6). Figure 10 runs the 0.5 rad turn with the controller seeing the heading and turn rate 0, 0.17, 0.3 and 0.33 s late. Watch the replies pull apart as the delay grows.

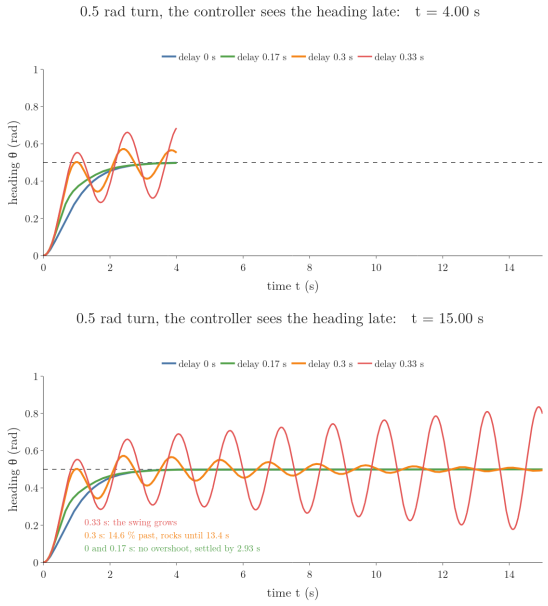


Figure 10: A time delay in the heading loop: the same 0.5 rad turn with PD gains 0.4 and 0.4, the controller seeing the heading and turn rate late. Blue, no delay, and green, 0.17 s: no overshoot, settled by 2.93 s. Orange, 0.3 s: 14.6 % past, rocking until 13.4 s. Red, 0.33 s: the swing grows

The numbers (notebook):

Delay	Overshoot	Settling time
0 s	0 %	2.92 s

Delay	Overshoot	Settling time
0.01 s (one 100 Hz step)	0 %	2.92 s
0.17 s (Autoware's)	0 %	2.93 s
0.3 s	14.6 %	13.4 s
0.33 s	grows	never

The mechanism. The controller acts on where the robot was, not where it is. While the robot swings towards the target, the delayed heading still shows it short, so the torque keeps pushing it on past; then it reads the overshoot late and brakes too long. Each correction comes later in the swing, until near 0.32 s it feeds the swing instead of damping it. Such a limit is the loop's delay margin, found from its frequency response; for this loop it is 0.324 s (notebook). A delay acts like a right-half-plane zero: it also delays the start of every reply and also limits how fast a loop can be made (FBS §10.4). Autoware's answer is to predict: it estimates where the car's speed will be after its 0.17 s delay from the acceleration it has already commanded, and runs its feedback on that prediction (`predictedStateAfterDelay` in `pid_longitudinal_controller.cpp`). The last trap is that the two loops share the robot's wheels.

10. SISO and MIMO: two wheels, two loops

> **Key point:** Every loop so far had one input and one output, a **SISO** system (G-3013), but the robot's two wheel forces each move both its speed and its heading: a **MIMO** system (G-3014), whose transfer function is a 2×2 table of fractions. A mixer that turns a force and a torque into wheel forces makes the table diagonal, so the speed and heading become two separate SISO loops.

The speed loop commanded a force and the heading loop a torque, as if each had its own motor. The robot has two wheels, and each wheel's force both pushes the robot forwards and turns it. The symbols:

- F_L and F_R are the left and right wheel forces, in N, each forward positive;
- the wheels are 0.2 m apart, so each sits 0.1 m from the centre (the differential drive);
- V and Θ are the speed's and the heading's multiples of the exponential, as in Section 2.

The forces add up to push the robot; their difference, times 0.1 m, turns it. With the transfer functions of Section 2:

$$V = \frac{F_L + F_R}{8s + 2}$$

$$\Theta = \frac{0.1\,(F_R - F_L)}{0.1\,s^2}$$

$$= \frac{F_R - F_L}{s^2}$$

As a table $G(s)$, one row per output (V, then Θ) and one column per input (F_L, then F_R):

$$G(s) = \begin{bmatrix} \frac{1}{8s+2} & \frac{1}{8s+2} \\ -\frac{1}{s^2} & \frac{1}{s^2} \end{bmatrix}$$

A system with one input and one output is **SISO** (G-3013), single-input single-output; one with several of each is **MIMO** (G-3014), multi-input multi-output (FBS §5.1). A MIMO system's transfer function is a table of fractions, one per input–output pair, and every entry here is non-zero: each wheel moves both outputs, as Figure 11 (a) draws.

The fix is a mixer: from the force u the speed loop asks for and the torque τ the heading loop asks for, compute the wheel forces. They must add up to the force, and their difference times 0.1 m must give the torque:

$$F_R + F_L = u$$

$$F_R - F_L = 10\,\tau$$

Adding and subtracting the two lines, then halving:

$$F_R = u/2 + 5\tau$$

$$F_L = u/2 - 5\tau$$

Check with a force of 1 N and a torque of 0.1 N · m (notebook):

$$F_R = 0.5 + 5 \times 0.1 = 1 \text{ N}$$

$$F_L = 0.5 - 5 \times 0.1 = 0 \text{ N}$$

$$F_R + F_L = 1 \text{ N}$$

$$(F_R - F_L) \times 0.1 = 0.1 \text{ N} \cdot \text{m}$$

which is the force and the torque asked for. Through the mixer the force moves only the speed, $1/(8s+2)$, and the torque turns only the heading, $10/s^2$: the table is diagonal, and the speed and heading loops of this chapter are two separate SISO loops (Figure 11 (b)). Real stacks do the same at the level of speeds: ros2_control's `diff_drive_controller` turns the commanded linear and angular velocity into the two wheel speeds, velocity_left = (linear −

angular × wheel_separation / 2) / left_wheel_radius and the same with a plus for the right (`diff_drive_controller.cpp`, lines 332 to 335).

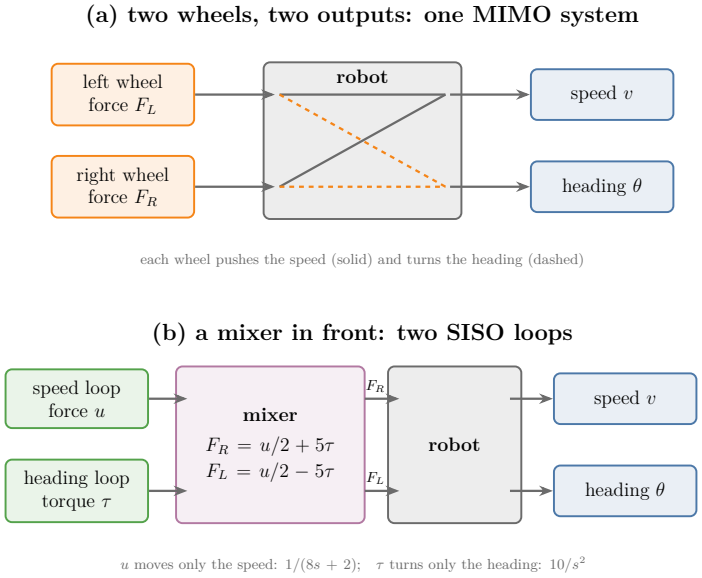


Figure 11: Two wheels, two outputs. (a) The robot as one MIMO system: the left and right wheel forces each push the speed (solid lines) and turn the heading (dashed lines). (b) A mixer in front, F_R = u/2 + 5τ and F_L = u/2 − 5τ, turns the speed loop's force u and the heading loop's torque τ into wheel forces; then u moves only the speed, 1/(8s + 2), and τ turns only the heading, 10/s^2: two SISO loops

11. Summary

The symbols: s the rate of an exponential input e^{st}; K_p, K_d, K_i the heading loop's P, D and integral gains (0.4, 0.4 and 0.2); T_d a time delay in s.

Idea	Formula or value	Why it matters
transfer function	speed $1/(8s+2)$, heading $10/s^2$	a system becomes one fraction of s
from state space	$C(sI-A)^{-1}B+D$	same fraction; poles = eigenvalues of A
poles	roots of the denominator	stability and shape of the reply
zeros	roots of the numerator	reshape the reply: -1 gives 13.5 % overshoot
BIBO stability	every pole in the left half-plane (no delay)	a bounded push gives a bounded reply
feedback rule	$PC/(1+PC)$	a loop's fraction without its equation
root locus	poles as K_i goes 0 to 2	cross at $\pm 2j$ when $K_i = 1.6$
Routh–Hurwitz	$a_1 a_2 > a_3$ for a cubic	stability limit from coefficients alone
time delay	e^{-sT_d}	no poles; 0.3 s makes the turn overshoot

- With the dragging gearbox the robot drifted 0.72 m to the right in 15 m with PD alone because a P loop needs a 0.05 rad error to push back 0.02 N · m; an integral gain of 0.2 removes that error within 4.68 s, and any gain up to 1.6 is stable, because the Routh–Hurwitz criterion on $s^3 + 4s^2 + 4s + 10K_i$ demands $16 > 10\ K_i$; a gain of 2 weaves because two poles have crossed into the right half-plane.
- A transfer function is the number a linear system multiplies an exponential e^{st} by, so a derivative becomes a factor s and a differential equation becomes a fraction we can combine by algebra.
- The state-space formula $C(sI-A)^{-1}B+D$ gives the same fraction, because an exponential input makes every state an exponential, so the poles are the eigenvalues of A.
- Each pole adds a term e^{pt} to every reply, so poles further left fade faster, poles further from the real axis swing faster and poles in the right half-plane grow.
- A zero adds a multiple of the reply's rate of change, so the same critically damped poles with a zero at -1 overshoot 13.5 %, and the second-order formulas hold only without zeros.
- A system without delay is BIBO stable exactly when every pole is in the left half-plane, so the heading plant alone (poles at 0) spins up under a steady 0.02 N · m, and a P-only loop (poles ±2j) grows under a wobble at 2 rad/s.
- Series blocks multiply, parallel blocks add and a loop is forward over one plus loop, so the PID loop reduces in four moves, and every transfer function of one loop shares its

denominator.

- Cancelling the speed plant's slow pole gives a clean reply to the command but keeps that pole in the reply to a push, so the robot needs 5.89 s instead of the 3.14 s of the usual integral gain of 10 to recover from the 1 % ramp; cancelling an unstable pole hides a signal that grows.
- A right-half-plane zero makes a reply start the wrong way, so the reversing robot's front bumper first moves 0.016 m the wrong way.
- A time delay e^{-sT_d} has no poles, so stability must be checked another way: the turn tolerates 0.17 s, overshoots at 0.3 s and swings wider at 0.33 s.
- The robot's two wheels make a MIMO system, so a mixer that turns a force and a torque into wheel forces is what lets the speed and heading be tuned as two SISO loops.

We can now write any linear loop of the robot as a fraction, read its stability and shape from its poles and zeros, and find the gain at which it goes unstable. Still missing: a delay has no poles, and real gains need a margin from the edge, not just a yes or no. The loop's reply to steady swings at each frequency, frequency response, handles both.

12. Sources

Built from

- Northwestern Robotics, "Modern Robotics, Chapter 11.3: Motion Control with Velocity Inputs (Part 2 of 3)", YouTube, https://www.youtube.com/watch?v=NAXmXg1gCI0
- Northwestern Robotics, "Modern Robotics, Chapter 11.4: Motion Control with Torque or Force Inputs (Part 2 of 3)", YouTube, https://www.youtube.com/watch?v=992hdsGgVlE
- Northwestern Robotics, "Modern Robotics, Chapter 11.2.2: Linear Error Dynamics", YouTube, https://www.youtube.com/watch?v=1TY4tpSnjBg
- NPTEL IIT Madras, *Control Engineering*, "Transfer function and block diagram", YouTube, https://www.youtube.com/watch?v=kbjCGGTXqUo (NPTEL L8)
- NPTEL IIT Madras, *Control Engineering*, "Effect of zeros on system response", YouTube, https://www.youtube.com/watch?v=PhfVRAML9yo (NPTEL L41)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, §11.2.2, §11.3, §11.4; narration pages at https://modernrobotics.northwestern.edu (MR 11.2.2, 11.3, 11.4)
- Chen, X., *Introduction to Control Theory* (University of Washington), Lecture 2 "Block Diagram Simplification", Lecture 6 "First- and Second-Order Systems", Lecture 9 "The Effect of Zeros", Lecture 10 "Stability Analysis: the Routh-Hurwitz Approach", Lecture 13 "The Root Locus Technique", https://faculty.washington.edu/chx/teaching/fbintro/ (Chen L2, L6, L9, L10, L13)
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed., §2.2 (Routh–Hurwitz, eq. 2.18), §5.1 (SISO and MIMO), ch. 9 "Transfer Functions": §9.2 (exponential signals, eq. 9.4, Example 9.5 time delay, Table 9.1), §9.3, §9.4 (block diagrams), §9.5 (poles, zeros, pole/zero cancellations, Example 9.12 cruise control), §10.4 (Example 10.11, vehicle steering in reverse), §10.5 (BIBO), §12.5 (the root locus). Free: https://fbswiki.org (FBS)

Other references

- MATLAB, "What are Transfer Functions? | Control Systems in Practice", YouTube, https://www.youtube.com/watch?v=2Xl7–Df3g8 (the planned main source; its subtitles were not available, so nothing here is built from it)
- MATLAB, "What Are Non-Minimum Phase Systems? | Control Systems in Practice", https://www.mathworks.com/videos/control-systems-in-practice-part-6-what-are-non-minimum-phase-systems-1574922412196.html (an approved source; its transcript page could not be opened, so the non-minimum-phase section rests on Chen L9, FBS Example 10.11 and NPTEL L41)
- NPTEL IIT Madras, *Control Engineering*, "Introduction to stability", "Routh Hurwitz criterion" and "Root locus-I", YouTube, https://www.youtube.com/watch?v=Irxppc_LCUk, https://www.youtube.com/watch?v=cez4InLZ7Pw, https://www.youtube.com/watch?v=sUDoTw_Ll
- PX4 Autopilot, https://github.com/PX4/PX4-Autopilot: `src/modules/ mc_autotune_attitude_cor` `mc_autotune_attitude_control.cpp` (lines 189 and 190, the identified model's numerator and denominator) and `src/lib/pid_design/pid_design.hpp` ("the discrete-time plant transfer function"); read for what is quoted
- Autoware Universe, https://github.com/autowarefoundation/autoware_universe: `control/ autoware_pid_longitudinal_controller/ config/ autoware_pid_longitudinal_cont` (`delay_compensation_time: 0.17`, `ki: 0.1`, `max_i_effort: 0.3`), its `README.md` (time delay compensation) and `src/pid_longitudinal_controller.cpp` (`predictedStateAfterDel` read for the numbers quoted
- ros2_control, ros2_controllers, https://github.com/ros-controls/ros2_controllers: `diff_drive_controller/src/diff_drive_controller.cpp` (lines 332 to 335, the wheel speeds from the linear and angular velocity); read for the lines quoted

13. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Transfer function (control) (G-2999)	The number a linear system multiplies an exponential input e^(st) by, written as a function of s, such as G(s) = 1/(8s + 2) for the robot's speed and $10/s^2$ for its heading; it turns a differential equation into a fraction, so systems can be combined and read by algebra.
Pole (G-3000)	A value of s where a transfer function's denominator is zero, so the fraction blows up there, such as −2 (twice) for the PD heading loop; each pole adds a term e^(pt) to every reply, so its real part says fade or grow and its imaginary part how fast it swings.

Term	Meaning
Zero (of a transfer function) (G-3001)	A value of s where a transfer function's numerator is zero, such as -1 for a PD loop with the D part on the error; it does not change stability but reshapes the reply, adding a multiple of the reply's rate of change.
Pole-zero map (G-3002)	A plot of a transfer function's poles (crosses) and zeros (circles) in the plane of complex numbers, real part across and imaginary part up; where the marks sit shows at a glance whether the system is stable and how it replies.
Left half-plane (G-3003)	The part of the plane of complex numbers where the real part is negative, left of the imaginary axis; poles there add terms that fade, so a system whose poles all lie there is stable, while the right half-plane holds growing terms.
BIBO stability (G-3004)	Bounded-input, bounded-output stability: every input that stays within limits gives an output that stays within limits; for a transfer function without delay it holds exactly when every pole lies in the open left half-plane.
Marginally stable (G-3005)	Having poles on the imaginary axis and none to the right, such as a P-only heading loop with poles $\pm 2j$: its own motion neither fades nor grows, but an input at its swing rate makes it grow, so it is not BIBO stable.
Block-diagram algebra (G-3006)	The rules that reduce a block diagram to one transfer function: blocks in series multiply, blocks in parallel add, and a feedback loop becomes the forward path divided by one plus the product around the loop; they give a loop's fraction without writing its differential equation.
Closed-loop transfer function (G-3007)	The transfer function from a closed loop's command to its output, $PC/(1 + PC)$ for a controller C and a plant P, such as $(4s + 10K_i)/(s^3 + 4s^2 + 4s + 10K_i)$ for the PID heading loop; its poles are the loop's poles, shared by every input of the loop.

Term	Meaning
Root locus (G-3008)	The paths a closed loop's poles trace as one gain changes, such as the three poles of the PID heading loop as the integral gain rises from 0 to 2; it shows for every gain at once whether the loop is stable, how fast it is and how much it swings.
Routh–Hurwitz criterion (G-3009)	A test that decides from a polynomial's coefficients alone whether all its roots have negative real parts; for a cubic $s^3 + a1\ s^2 + a2\ s + a3$ it asks that all coefficients be positive and $a1\ a2 > a3$, which gives the integral-gain limit 1.6 of the heading loop.
Pole-zero cancellation (G-3010)	A controller zero placed exactly on a plant pole, so the pole drops out of the transfer function from the command, such as a PI zero at -0.25 on the speed plant's pole; the pole stays in the reply to disturbances, and cancelling an unstable pole hides a signal that grows.
Non-minimum phase (G-3011)	Describes a system with a zero in the right half-plane, or a time delay, such as the reversing robot's front point; a right-half-plane zero makes the step response start in the wrong direction (undershoot) and a delay makes it start late, and both limit how fast a loop around it can be made.
Time delay (dead time) (G-3012)	A wait of T seconds between a signal and its effect, such as a camera reporting the heading late, with transfer function e^(−sT); it has no poles or zeros but limits the gains a loop can use, here making the turn swing for ever near 0.32 s.
SISO system (G-3013)	A single-input single-output system, such as the heading loop from torque to heading; it has one transfer function, and all the tools of classical control apply to it directly.
MIMO system (G-3014)	A multi-input multi-output system, such as the robot with two wheel forces in and speed and heading out; its transfer function is a table of fractions, one per input–output pair, and a mixer can make that table diagonal so it splits into SISO loops.

Term	Meaning
Disturbance (G-2949)	An unwanted outside effect that changes what the plant does, such as a ramp pulling a robot back with 0.785 N; it changes the true output, unlike sensor noise, which changes only the reading.
Integral action (G-2963)	A part of the command in proportion to the error's running sum (its integral over time); the sum grows while any error remains, so it removes the error a steady load leaves, at the cost of overshoot and, if too large, instability.
Integral gain (G-2964)	The factor K_i by which integral action multiplies the error's running sum, such as 0.05 N m per rad s; larger values remove a steady error sooner but swing more, and above a limit make the loop unstable.
DC gain (G-2989)	The ratio of a stable system's final output to the size of a step input, such as 0.75 for the speed loop (0.375 m/s for a 0.5 m/s command); it says how much of a steady command the output ends up delivering.
Characteristic equation (G-2993)	The polynomial equation found by trying a solution e^(st) in a linear differential equation, such as $s^2 + 0.8\ s + 4 = 0$ for the heading error; its roots say whether the solution fades, swings or grows.
Eigenvalue (G-665)	The number by which a matrix stretches or shrinks one of its eigenvectors: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvalues of the covariance matrix are the variances along the principal components.
Characteristic polynomial (G-376)	A polynomial built from a square matrix A, $\det(A - \lambda I)$, whose roots are the eigenvalues of A, so solving it finds them.
Block diagram (G-2950)	A picture of a control system with each part drawn as a box and each signal as an arrow between them, with circles where signals add or subtract; it shows where every signal, disturbance and noise enters the loop.

Summing junction (G-2951)	A circle in a block diagram where the arriving signals are added or subtracted, each with the sign written beside it, such as reference minus measured output giving the error.